

Network Telemetry Fault Detection & Root Cause Attribution

Project Item	Summary
Data size	10 cell sites, 21 days, one reading every 15 minutes
Main task	Find abnormal network behaviour and suggest likely root cause
Model type	Unsupervised anomaly detection; labels are not used for training
Final output	Anomaly alerts, top affected KPIs, predicted fault type, evaluation metrics

1. Executive Summary

This project simulates the monitoring of mobile network cell sites. Each site produces regular KPI readings such as traffic, users, call success, drop rate, signal quality, voltage, temperature, and latency. The aim is to detect early signs of faults before they become customer-impacting issues.

To start with, a realistic synthetic dataset was created because there was no real network data available to work with. Next, faults were intentionally added to the data in a controlled manner, allowing for a more accurate test of the system. The detection system was trained using only the first 10 days of telemetry data, and then it was tested on the data from the later period. During the test, the system successfully produced anomaly alerts, ranked the most responsible KPIs, and even predicted the most likely type of fault that had occurred. This process helped to evaluate the effectiveness of the detection system in identifying and responding to faults in the network.

The main focus of the solution is not to claim perfect accuracy, but to show a practical and honest workflow: generate realistic data, avoid label leakage, compare against ground truth only during final evaluation, and explain both strengths and failure cases.

2. Problem Statement

Telecom operators receive thousands of performance readings from cell sites every day. Traditional threshold alarms can be noisy because normal traffic changes between day and night, weekdays and weekends, and urban and rural locations. As a result, engineers may receive too many alerts and may miss early signs of real faults.

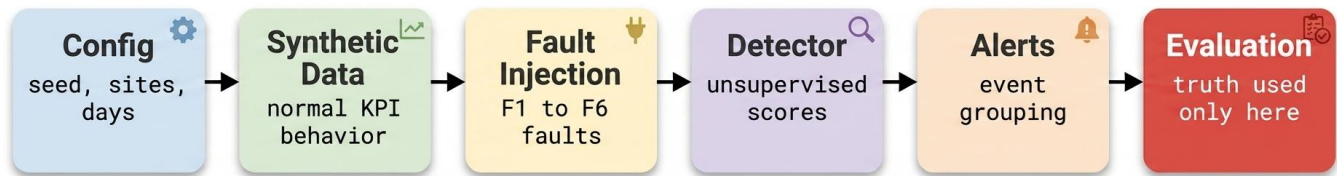
So, the goal of this project is to figure out when something's not quite right by looking at lots of different performance indicators all at once. Then, it tries to give a straightforward explanation for what might be causing the problem. This way, we can get to the bottom of things quickly and easily.

3. Objectives

- Create a realistic synthetic telemetry dataset for 10 cell sites over 21 days.
- Inject all six required fault types in a physically meaningful way.
- Train an unsupervised detector using only the training window.
- Convert anomaly scores into practical alerts while controlling false alarms.
- Provide root-cause hints by ranking the top affected KPIs and predicting a fault type.
- Evaluate honestly using event-level and point-level metrics.

4. Methodology Overview

Program Methodology



» Labels are kept separate: detector sees telemetry only; evaluator reads ground truth at the end.

Figure 1: Overall workflow used in the project.

The workflow starts with configuration and synthetic data generation. After that, faults are injected and saved separately as ground truth. The detector reads only telemetry data, creates anomaly scores and alerts, and then the evaluation script compares the detected events with the hidden ground truth.

5. Dataset Design

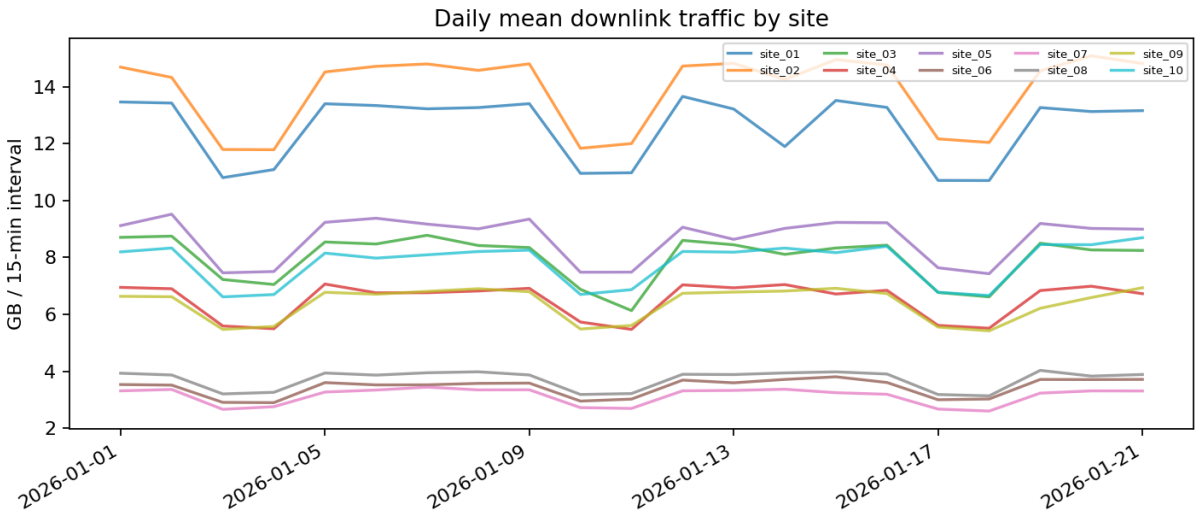
The dataset represents mobile network cell sites. Each row is one site at one timestamp. The interval is 15 minutes, which means each site has 2,016 readings over 21 days.

Feature	Design Choice
Sites	10 cell sites with different behaviour patterns
Duration	21 days
Interval	15 minutes
KPIs	12 correlated network KPIs
Training period	Days 1 to 10
Evaluation period	Days 11 to 21
Missing values	Small random missingness to mimic real telemetry issues

The normal behaviour was designed to look realistic. Traffic is lower at night and higher during busy hours. Weekends are not exactly the same as weekdays. Urban sites are busier than rural sites. KPIs also move together: for example, when users and traffic increase, CPU load and latency can also increase.

KPI Group	Examples
Traffic and usage	Downlink traffic, active users
Service quality	Call setup success, call drop rate, handover success
Radio condition	Signal quality, retransmission rate, uplink interference
System health	CPU load, unit temperature, supply voltage, latency

Figure 2: Example site traffic patterns showing that sites do not behave identically.



6. Fault Injection Strategy

Faults were injected after the normal dataset was generated. Each fault affects several KPIs together, because real telecom issues usually do not appear as a single isolated spike. The purpose was to create faults that are detectable but not artificially easy.

Fault	Simple Meaning	Expected KPI Behaviour
F1 RF degradation	Radio unit slowly weakens	Signal quality drops; drops and retransmissions increase
F2 Clock sync drift	Timing issue	Handover success slowly degrades; other KPIs remain near normal
F3 Power instability	Power supply issue	Voltage oscillates; CPU and temperature become unstable
F4 Sleeping cell	Site carries almost no traffic	Traffic and users collapse while some quality KPIs look normal
F5 Interference burst	External interference	Interference rises sharply; quality and success rates worsen
F6 Capacity overload	Heavy real usage	Traffic surge causes latency and drop-rate pressure

7. Detection Approach

The detector is an unsupervised one. In other words, it doesn't learn from labeled samples of faulty data or normal data. It learns what normal behavior is based on the first ten days and then decides whether further readings are anomalies.

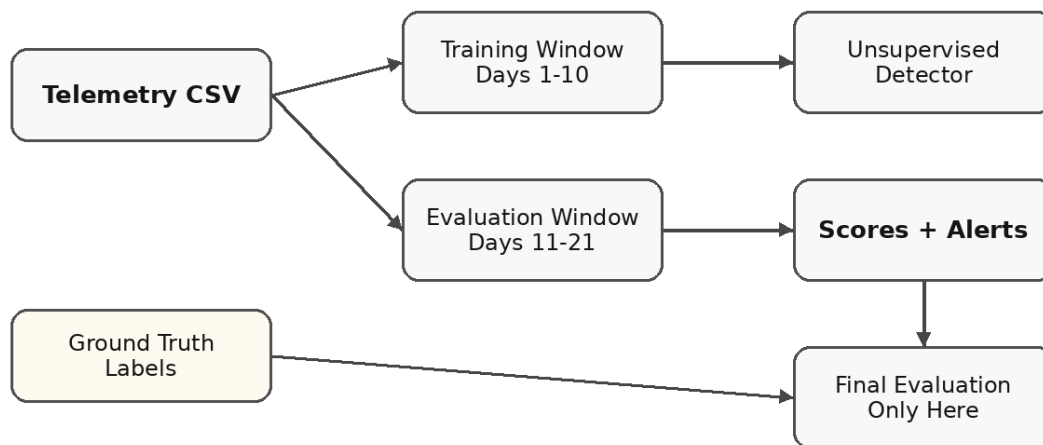
Prior to training, missing values for KPIs are imputed using interpolation and forward/backward fill techniques. Moreover, time-related features are added since the detector should understand the difference between nighttime traffic and busy hours traffic.

The final anomaly scoring consists of three fairly simple heuristics. The Isolation Forest finds abnormal readings; PCA reconstruction error finds abnormal relationships between KPIs; and robust z-score baseline finds extreme readings.

Component	Reason for using it
Isolation Forest	Finds unusual rows compared to normal data
PCA reconstruction error	Detects when KPI relationships no longer look normal
Robust z-score baseline	Simple comparison method for obvious deviations
Per-site threshold	Each site has its own normal range, so alerts are fairer

8. Leak-Free Evaluation Setup

Leak-Free System Architecture



The detector never uses labels. Labels are used only to measure final performance.

Figure 3: Ground truth is separated from the detection path.

The most important rule followed in the project is that the model, threshold selection, and root-cause logic do not read the ground truth folder. The ground truth is used only by the evaluation script after predictions are already created. This avoids data leakage and makes the final scores more trustworthy.

9. Root Cause Attribution

After an alert is detected, the system checks which KPIs changed the most during that period. It then reports the top three responsible KPIs and predicts the closest fault archetype using simple rules. For example, a traffic and user collapse suggests a sleeping cell, while high interference with poor signal quality suggests an interference burst.

10. Results and Interpretation

Metric	Value	Plain-English Meaning
Evaluation fault events	27	Number of real test-window faults
Detected events	59	Number of alert events produced
Event precision	0.356	How many detected events were real
Event recall	0.778	How many real faults were found
Event F1	0.488	Balance of precision and recall
Point-level F1	0.250	Strict timestamp-level performance
Point-adjust F1	0.914	More generous anomaly metric
False alerts / site / day	1.573	Kept below the 2-alert budget
Attribution hit-rate	1.000	Top-3 KPIs usually included a true affected KPI

The classifier was able to detect most of the true events, as demonstrated by the event recall. The precision is less due to the detector also producing additional alert events. This is plausible when detecting anomalies unsupervisedly, as the choice of threshold did not depend on test labels.

F1 adjusted for points is significantly larger compared to F1 computed strictly on the point level. This is quite natural since adjusted F1 assigns points for an entire fault period when just one point inside the window has been detected. For practical use, however, other measures such as event recall and delay are more relevant.

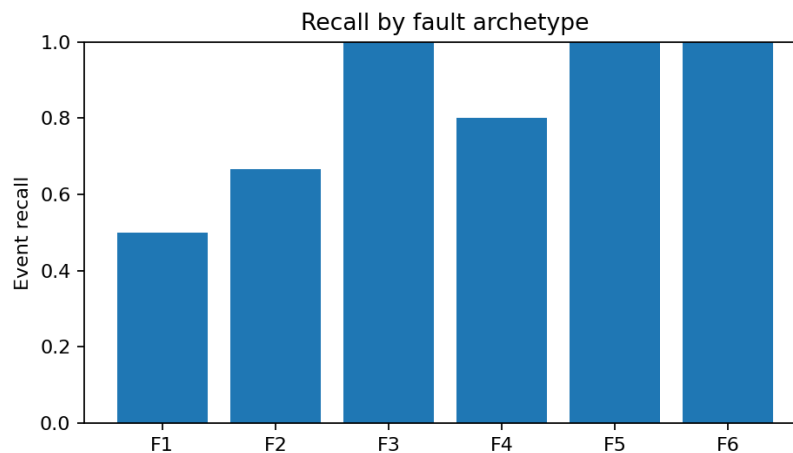


Figure 4: Recall by fault type. Some subtle faults are harder than sudden faults.

Fault	Events	Recall	Mean Delay (min)
F1	4	0.50	524.2
F2	9	0.67	1000.0
F3	5	1.00	51.7
F4	5	0.80	43.3
F5	3	1.00	4.5
F6	1	1.00	129.4

11. Failure Case Study

A challenging example is handover drift (F2), where handover success is degraded slowly, whereas most other KPIs can stay nearly unchanged from their usual levels. Since this degradation is gradual and has only an indirect impact on other KPIs, the detection of this anomaly is likely to occur later than others.

This is because gradual issues such as timing problems are hard to detect compared to sudden interference events or traffic breakdowns. A more advanced model should incorporate a forecasting engine for detecting anomalies related to handover success or a comparison of neighbouring cells.

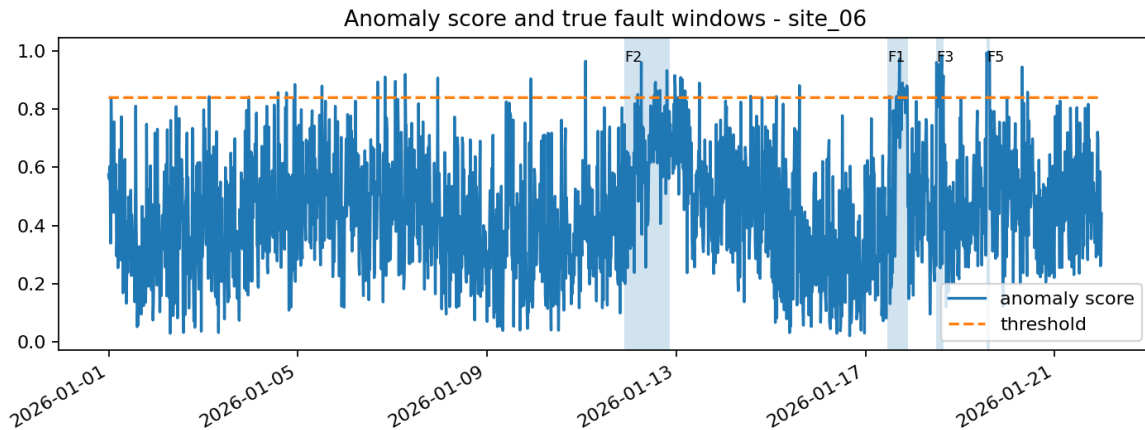


Figure 5: Example anomaly score and threshold plot used to inspect detection behaviour.

12. Limitations

- The data is synthetic, so it cannot capture every detail of a real telecom network.
- The fault classifier is rule-based and may struggle with mixed or overlapping faults.
- Subtle gradual faults can be detected late because they do not produce strong KPI changes immediately.
- The system does not use external context such as local events, weather, maintenance logs, or neighbouring cell behaviour.
- Thresholds may need adjustment before production use on real data.

13. Next Steps

- Add forecasting-based detection to catch slow changes earlier.
- Use neighbouring-site comparison to distinguish local faults from area-wide traffic changes.
- Improve the root-cause mapper using more rules or weakly supervised training.
- Add severity scores so engineers can prioritize the most important alerts first.
- Build a simple dashboard for viewing alerts, responsible KPIs, and confidence levels.

14. Conclusion

This case study showcases a comprehensive pipeline for fault detection on networks using network telemetry. The process is started with the generation of synthetic data that is realistic, followed by the addition of telecom faults, training an unsupervised fault detector, translating scores to alerts, and reasoning about the cause of faults in the form of KPIs.

The process is purposefully transparent in that it does not shy away from highlighting good performance, poor performance, challenging fault detection cases, and areas of improvement. In conclusion, it showcases how AI/ML technologies can be used in support of network management by easing the monitoring burden.